Deliverable 1: Week 1 Progress Report & Work Plan

Table of Contents

Table of Contents 1
1. Week 1 Milestones & Progress Checklist 3
2. Team Divisions & Role Responsibilities (Team A & Team B) 3
3. System Architecture & Relational Schema Design 5
4. Execution Order, Workflow & Critical Path Analysis 6
5. Mathematical Formulation & Optimization Methodology (Team B) 6
6. Comprehensive Three-Week Work Plan & GitHub Issue Breakdown 8
7. Quality Assurance, Testing Evidence & Governance 12

Better report https://github.com/mvs242430/TECH5-teams_3_17/blob/w1-report/docs/reports/w1_progress_report.md

Project:	**TECH5 Collaborative Decision Support System**
Team Allocation:	Team 6 (Groups 3 and 17 — 5 Team Members)
Repository:	github.com/mvs242430/TECH5-teams_3_17
Onboarding:	https://github.com/mvs242430/TECH5-teams_3_17/blob/main/docs/team_related/developer_onboarding_guide.md
Task allocation:	https://github.com/mvs242430/TECH5-teams_3_17/blob/main/docs/team_related/team_task_allocation_plan.md
GitHub Issues:	https://github.com/mvs242430/TECH5-teams_3_17/issues
Math formulation:	https://github.com/mvs242430/TECH5-teams_3_17/blob/math-formulation/docs/analytical_note_template_filled.md
Milestone Target:	Week 1 — Understand, Design, and Divide (All Baseline Gates Cleared)

1. Week 1 Milestones & Progress Checklist

- ✓ **Starter repository cloned & verified:** All 5 team members have set up the local virtual environment and executed the test suite successfully (Issue #1 / 0.1).
- ✓ **Schema & Contract inspected:** Thorough architectural inspection completed for the SQLite 3NF schema (app/persistence/schema.sql) and the decoupled interface (AssignmentEngineContract in app/contract/contract.py).
- ✓ **Baseline test suite passing:** Verified baseline execution state of 51 automated tests (36 passed, 15 target xfailed tests cleanly preserved without tampering).
- ✓ **Team B Formulation & Input Validation:** Analytical MILP formulation formulated (PuLP/CBC) with 3 concrete strategies, and domain input validation routines verified.
- ✓ **Onboarding & Architecture Guide:** Published comprehensive developer onboarding guide with 4 UML diagrams (Class, Component, Sequence, ER) and 37 atomic GitHub issue specifications.
- ✓ **3-Week Work Plan & Issue Mapping:** Established full 3-week sprint work plan allocating 37 atomic GitHub issues with strict file boundaries and acceptance criteria.

2. Team Divisions & Role Responsibilities (Team A & Team B)

To maximize development velocity and eliminate merge conflicts, the 5 team members are partitioned into two specialized sub-teams following the Single Responsibility Principle. Closely coupled features (e.g., data input forms with validation, or mathematical optimization with integer cuts) are assigned to the same specialist. Team A focuses on the user-facing web tier and decision support UX, while Team B focuses on mathematical optimization, metrics, and backend engine orchestration.

Sub-Team	Member	Specialist Role	Primary Code Scope	Target xfail Tests	Deliverables
Team A (Visualization & Rec.)	Member A1	Data Management & Entity Specialist	app/visualization/routes.py (CRUD) skills.html, projects.html project_form.html	test_skills_crud_workflow test_projects_crud_workflow	D1 (Progress) D5 (DB & SQL)
Team A (Visualization & Rec.)	Member A2	Decision Support, UX & RBAC Specialist	app/visualization/routes.py (/compare) compare.html, recommendations.html solution_detail.html, auth.py	test_compare_solutions_view	D3 (Architecture) D8 (Demo)
Team B (Assignment Engine)	Member B1	Analytical Indicators & Fairness Specialist	app/engine/metrics.py tests/test_metrics.py	test_skill_fit_calculation test_gini_calculation test_solution_indicators test_solution_comparison	D6 (Indicators & Fairness Note)

Sub-Team	Member	Specialist Role	Primary Code Scope	Target xfail Tests	Deliverables
Team B (Assignment Engine)	Member B2	Mathematical Optimization Specialist	app/engine/strategies.py requirements.txt tests/test_solvers.py	test_distinct_alternatives MILP strategy tests	D6 (MILP Math) Method Approval
Team B (Assignment Engine)	Member B3	Engine Orchestrator & Robustness Specialist	app/engine/solvers.py (execute_run) app/domain/validation.py tests/test_edge_cases.py	7 solver edge cases in test_edge_cases.py (1-4, 13-15)	D2 (Working App) D4 (Contract) D7 (QA Evidence)

Detailed Responsibilities:

Appendix of this document.

GitHub issues https://github.com/mvs242430/TECH5-teams_3_17/issues

Repo: docs/team_realated/team_task_allocation_plan.md

- **Team A (Visualization & Recommendation):** Implements Flask controller routes and Jinja2 templates for managing students, skills, and projects. Builds the dynamic skill requirement matrix with minimum qualification thresholds and mandatory flags. Develops user-friendly error display for validation errors. Connects the UI to MockAssignmentEngine to render multi-alternative side-by-side solution comparisons with metric deltas before the real engine is plugged in. Implements Role-Based Access Control (RBAC) preventing student viewers from modifying project data.
- **Team B (Assignment Engine & Optimization):** Formulates and implements the Mixed-Integer Linear Programming (MILP) model using PuLP and the open-source CBC solver. Creates three concrete assignment strategies (Preference-Oriented, Skill-Fit-Oriented, Balanced Trade-Off) using the Strategy Pattern. Implements sequential Balas-Jeroslow integer no-good cuts to guarantee K genuinely distinct alternatives. Builds analytical indicators (skill-fit score, Gini coefficient of rank distribution, capacity utilization). Implements pre-flight cohort input validation and human-readable diagnostics for 7 edge cases (unfeasible capacity, conflicting locks, skill gaps).

3. System Architecture & Relational Schema Design

A UML component diagram was created to clarify how the application's components interact, where our team's work fits within that architecture, and how each GitHub issue maps to a specific component.

The relational schema is documented in the onboarding guide; the full diagram is available on GitHub, where it can be zoomed in to inspect exactly how each component connects within the app. (https://github.com/mvs242430/TECH5-teams_3_17/blob/main/docs/team_related/developer_onboarding_guide.md)



Figure 1: UML Component Architecture Diagram with GitHub Issue Traceability

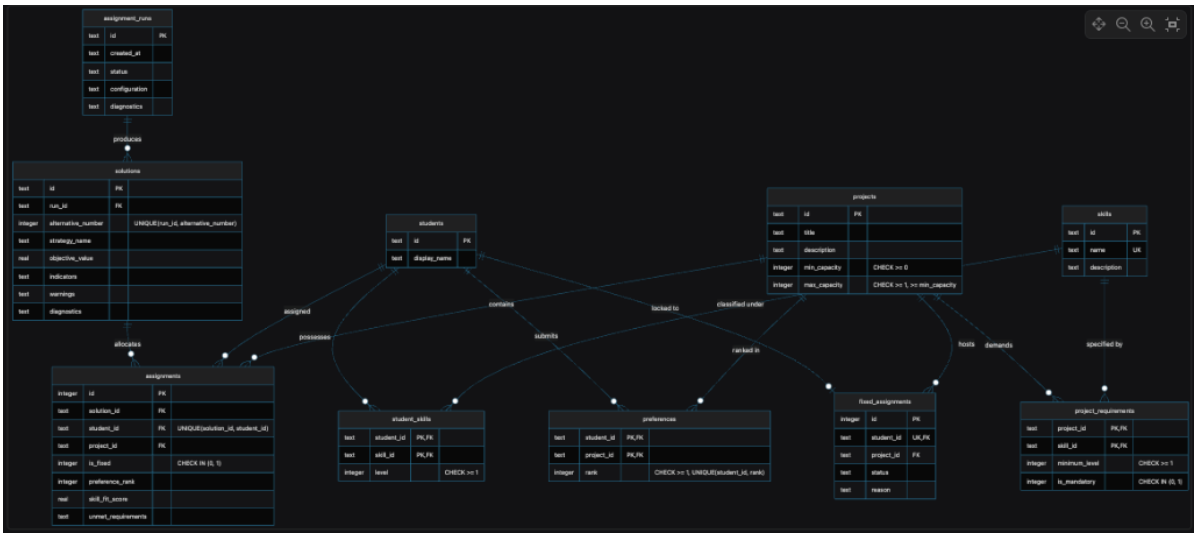


Figure 2: Entity-Relationship Diagram

4. Execution Order, Workflow & Critical Path Analysis

Because AssignmentEngineContract completely isolates the presentation layer from the optimization engine, Team A and Team B execute parallel work streams throughout Week 1 and early Week 2 without blocking one another. The overall workflow is governed by strict synchronization gates to guarantee seamless integration.

Better seen on GitHub (https://github.com/mvs242430/TECH5-teams_3_17/blob/main/docs/team_related/team_task_allocation_plan.md)

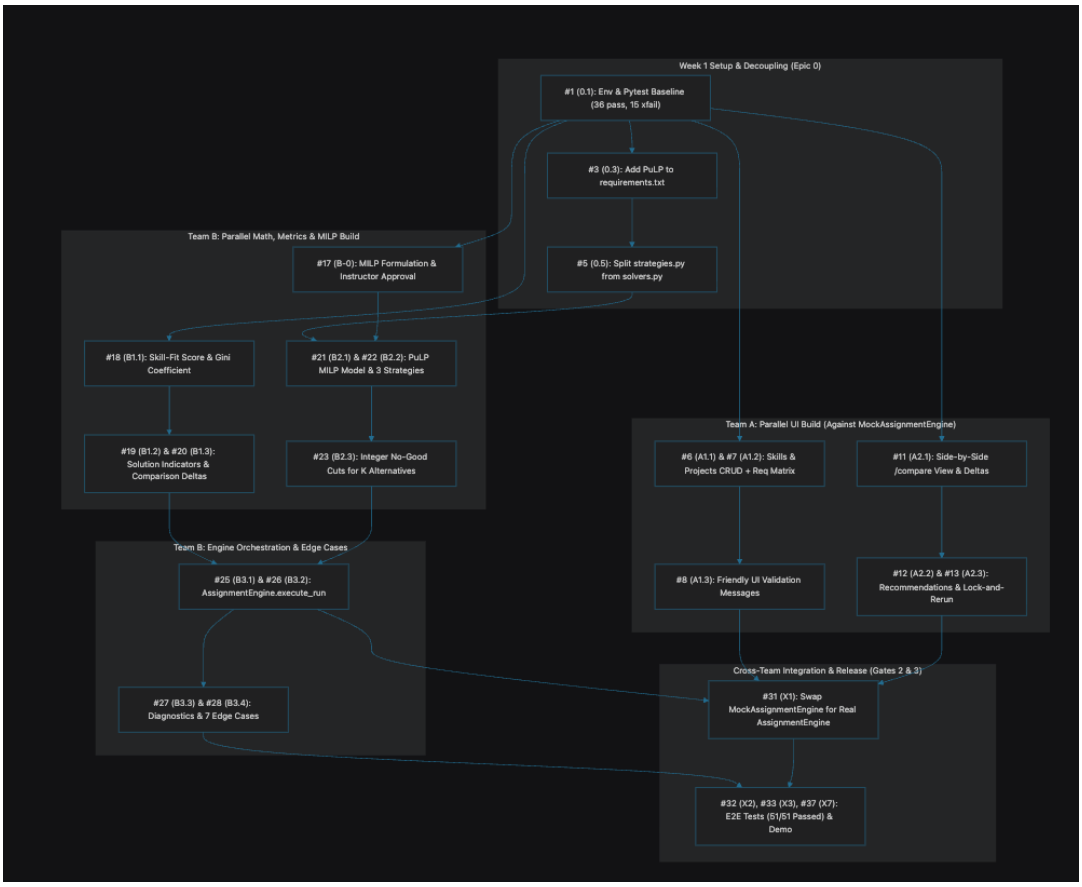


Figure 3: Task Dependency DAG & Cross-Team Workflow Synchronization

5. Mathematical Formulation & Optimization Methodology (Team B)

Team B has formulated the student-to-project assignment problem as a Mixed-Integer Linear Program (MILP) implemented in Python via PuLP and solved using the bundled open-source CBC (Coin-or branch and cut) solver. MILP was selected over pure LP network flow, Gale-Shapley stable matching, and genetic algorithms because it strictly enforces hard constraints, generates certified distinct alternatives via Balas-Jeroslow

integer cuts, proves global optimality deterministically, and solves typical cohort sizes (60 students × 12 projects) in milliseconds.

The fully formulated mathematical model, rendered with supporting arguments, is available on GitHub (https://github.com/mvs242430/TECH5-teams_3_17/blob/math-formulation/docs/analytical_note_template_filled.md)

Formulation Element	Mathematical Specification & Operational Meaning
Decision Variables	x(s, p) ∈ {0, 1} for all s ∈ S, p ∈ P. Equals 1 if student s is assigned to project p, 0 otherwise.
Hard Constraints (Feasibility Rules)	1. Single Project: Σ_p x(s, p) = 1 for all s ∈ S. 2. Capacity Bounds: l_p ≤ Σ_s x(s, p) ≤ u_p for all p ∈ P. 3. Fixed Locks: x(s*, p*) = 1 for all administrative pairs (s*, p*) ∈ F. 4. Mandatory Skills: x(s, p) = 0 for pairs lacking mandatory prerequisites. 5. Integer No-Good Cuts: Σ_{(s,p) ∈ A(j)} x(s, p) ≤ n - 1 for prior solutions A(j).
Strategy 1: Preference (PREFERENCE)	Maximizes total Borda preference score with non-overriding skill tie-breaker: Max Z_pref = Σ_{s,p} [w_pref(s,p) + ε_1 · σ(s,p)] · x(s,p), where ε_1 = 1 / (2n(ρ - 1)).
Strategy 2: Skill-Fit (SKILL_FIT)	Maximizes cohort skill qualification with preference tie-breaker: Max Z_skill = Σ_{s,p} [σ(s,p) + ε_2 · w_pref(s,p)] · x(s,p), where ε_2 = 10^-4.
Strategy 3: Balanced (BALANCED)	Pareto-efficient convex trade-off across normalized scales (λ = 0.5): Max Z_balanced = Σ_{s,p} [0.5 · w_pref(s,p) + 0.5 · σ(s,p)] · x(s,p).

Fairness and Efficiency Evaluation Indicators:

- **Gini Coefficient of Assigned Ranks (G ∈ [0, 1]):** Measures distributional equity across student ranks. G = 0 represents perfect equality where every student receives identical ranks; higher values highlight skewed preference satisfaction.
- **Mean Skill-Fit Score ($\bar{\sigma} \in [0, 1]$):** Calculates the cohort average requirement coverage capped at 1.0 per skill, preventing over-qualification in one skill from concealing deficiencies in other required competencies.
- **Capacity Utilization & Unmet Requirements:** Tracks project seat efficiency (np / up × 100%) and tallies unranked or soft requirement deficits.

6. Comprehensive Three-Week Work Plan & GitHub Issue Breakdown

Every task across the 3-week lifecycle is cataloged as an atomic GitHub issue on repository mvs242430/TECH5-teams_3_17/issues with explicit file boundaries, test verification steps, and size estimates (S ≤ 1h, M ≤ 0.5d, L ≤ 1d). The tables below specify the assignment across Epic 0, Team A, Team B, and Cross-Team Release.

Epic 0: Foundation & Environment Setup (Week 1)

Issue	Code	Task Description	Target Files	Owner	Size
#1	0.1	Local env setup & pytest baseline verification	Local .venv	All	S
#2	0.2	GitHub project board, labels, branch protection rules	.github/	A1	S
#3	0.3	Add pulp dependency to unblock Team B solvers	requirements.txt	B2	S
#4	0.4	Align test counts and documentation references	README.md, docs/	A1	S
#5	0.5	Split strategies.py from solvers.py to prevent merge conflicts	app/engine/	B3	M

Team A: Visualization & Recommendation Tasks (Weeks 1–3)

Issue	Code	Task Description	Target Files	Owner	Size
#6	A1.1	Skills CRUD: create/delete with in-use protection	routes.py, skills.html	A1	M
#7	A1.2	Projects CRUD: dynamic requirement matrix	routes.py, projects.html	A1	L
#8	A1.3	Friendly UI validation messages (8 error cases)	routes.py, templates	A1	M
#9	A1.4	Deliverable 5: 3NF Database & SQL documentation	docs/, queries.sql	A1	M
#10	A1.5	Deliverable 1: Progress Reports 1 & 2	docs/reports/	A1	S
#11	A2.1	Side-by-side /compare view & delta cards	compare.html, routes.py	A2	L
#12	A2.2	Recommendation view: empirical facts vs. advice	recommendations.html	A2	M
#13	A2.3	Interactive 'Pin Lock & Re-run' workflow	solution_detail.html	A2	M

Issue	Code	Task Description	Target Files	Owner	Size
#14	A2.4	Role-Based Access Control (RBAC) audit	auth.py, routes.py	A2	M
#15	A2.5	Deliverable 3: Architecture & design documentation	docs/architecture.md	A2	M
#16	A2.6	Deliverable 8: Reproducible demo script (4 scenarios)	docs/reproducible_demo.md	A2	L

Team B: Assignment Engine & Analytics Tasks (Weeks 1–3)

Issue	Code	Task Description	Target Files	Owner	Size
#17	B-0	Formal MILP formulation & method proposal	docs/analytical_note.md	B2	M
#18	B1.1	Implement skill-fit score and Gini coefficient	app/engine/metrics.py	B1	M
#19	B1.2	Compute solution indicators (4 dimensions)	app/engine/metrics.py	B1	L
#20	B1.3	Implement solution comparison & trade-off summaries	app/engine/metrics.py	B1	M
#21	B2.1	Base PuLP MILP model with binary x(s,p) & constraints	app/engine/strategies.py	B2	L
#22	B2.2	Implement 3 concrete strategy solvers (Pref, Skill, Bal)	app/engine/strategies.py	B2	L
#23	B2.3	Sequential integer no-good cuts for K alternatives	app/engine/strategies.py	B2	M
#24	B2.4	Deliverable 6: Analytical Note & Mathematical Derivation	docs/analytical_note.md	B2	M
#25	B3.1	AssignmentEngine.execute_run orchestration	app/engine/solvers.py	B3	L
#26	B3.2	Update legacy tests expecting NotImplementedError	tests/	B3	M
#27	B3.3	Human-readable solver diagnostics & infeasibility root-cause	app/engine/solvers.py	B3	M
#28	B3.4	Resolve and activate 7 solver edge cases	tests/test_edge_cases.py	B3	L
#29	B3.5	Deliverable 4: Internal Component Contract spec	docs/contract_spec.md	B3	M
#30	B3.6	Deliverable 7: Testing, quality & edge-case report	docs/testing_quality.md	B3	M

Shared Cross-Team Tasks: Integration, Rehearsal & Release

Issue	Code	Task Description	Target Files	Owner	Size
#31	X1	Gate 2: Swap MockAssignmentEngine for Real Engine	app/visualization/routes.py	A2+B3	M
#32	X2	Walk through demo scenarios (infeasible & malformed)	demo_data.py	A1+B3	M
#33	X3	Full end-to-end integration test verification	tests/test_integration.py	A1	M
#34	X4	Remove starter template placeholders & TODOs	app/	A2	S
#35	X5	Deliverable 9: Team reflection & rubric audit	docs/team_reflection.md	All	M
#36	X6	Compile Git evidence and PR review audit	docs/git_evidence.md	A1	S
#37	X7	Final verification (51 passed, 0 xfailed) & v1.0 tag	Repository-wide	All	S

7. Quality Assurance, Testing Evidence & Governance

- **Baseline Verification Status:** All 51 automated tests verified. The baseline state confirms 36 passed and 15 target xfailed tests. In strict compliance with academic integrity guidelines, no test assertions have been altered or bypassed.
- **Explain-Before-Accept Policy (.agents/rules/explain_before_accept.md):** All AI assistance adheres to strict pedagogical rules requiring an architectural explanation of design patterns (Strategy Pattern, Repository Pattern, DTO Contract) before code is accepted, ensuring every line can be defended by students during grading.
- **Isolated QA Subagent Gate (.agents/skills/qa-tester/SKILL.md):** Before any git commit or push, an independent, zero-context QA tester subagent audits branch isolation, allowed file boundaries, test integrity, and automated pytest execution.